Bước 40 — Rate Limiting & GraphQL Security

Ở Bước 39, chúng ta bảo vệ server khỏi một GraphQL query quá nặng.

Ở Bước 40, chúng ta giải quyết một vấn đề khác:

> Nếu client, bot hoặc attacker gửi quá nhiều GraphQL request thì sao?

Trong project Rails + GraphQL của bạn, nên hiểu bảo mật theo nhiều lớp:

```
CLIENT
  │
  │ POST /graphql
  ▼
Rack::Attack
  │
  │ Rate Limiting
  ▼
GraphqlController
  │
  ▼
GraphQL Schema
  │
  ├── Depth Limit
  ├── Complexity Limit
  └── GraphQL Validation
  │
  ▼
Authentication
  │
  ▼
Policy / Authorization
  │
  ▼
Resolver / Mutation
  │
  ▼
Service
  │
  ▼
Database
```

1. Rate Limiting là gì?

Rate limiting giới hạn:

```
ai đó được gọi API bao nhiêu l□n
trong một kho□ng thời gian
```

Ví dụ:

```
100 requests / 1 minute / IP
```

Nghĩa là:

```
IP: 84.123.10.20

request 1    → OK
request 2    → OK
...
request 100  → OK

request 101  → BLOCK
```

HTTP response:

```
429 Too Many Requests
```

2. Tại sao GraphQL cần Rate Limiting?

REST thường có nhiều endpoint:

```
GET /projects
GET /tasks
POST /tasks
DELETE /tasks/10
```

GraphQL thường chỉ có:

```
POST /graphql
```

Nhưng bên trong /graphql có thể là:

```
query {
  me {
    id
  }
}
```

hoặc:

```
query {
  projects {
    nodes {
      tasks {
        nodes {
          assignee {
            projects {
              nodes {
                tasks {
                  nodes {
                    id
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Cả hai đều là:

```
POST /graphql
```

nhưng chi phí hoàn toàn khác nhau.

Do đó GraphQL cần ít nhất hai hướng bảo vệ:

```
Rate Limiting
→ ch□ng quá nhi□u request

Complexity / Depth
→ ch□ng một request quá nặng
```

3. Công cụ chúng ta dùng

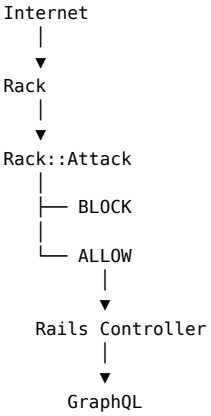
Trong Rails:

```
Rack::Attack
```

Nó nằm trước:

```
GraphqlController
```

Flow:



Đây là điểm rất quan trọng:

> Request bị rate limit sẽ bị chặn trước khi Resolver, Service và SQL chạy.

4. Cài rack-attack

Mở:

```
Gemfile
```

thêm:

```
gem "rack-attack"
```

Sau đó:

```
bundle install
```

5. Kiểm tra middleware

Có thể chạy:

```
bin/rails middleware
```

Bạn cần thấy:

```
Rack::Attack
```

trong middleware stack.

6. Tạo initializer

Tạo chính xác file:

```
config/initializers/rack_attack.rb
```

Ban đầu:

```
# frozen_string_literal: true

class Rack::Attack
end
```

Bây giờ chúng ta thêm từng lớp bảo vệ.

7. Rule đầu tiên: /graphql theo IP

Trong:

```
config/initializers/rack_attack.rb
```

thêm:

```
class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    if request.post? &&
       request.path == "/graphql"
      request.ip
    end
  end
end
```

Ý nghĩa:

```
"graphql/ip"
→ tên rule

limit: 100
→ t□i đa 100 requests

period: 1.minute
→ trong 1 phút

request.ip
→ đ□m riêng từng IP
```

8. Tại sao block phải return request.ip?

Đây là phần rất quan trọng.

Rack::Attack cần biết:

> Tôi đang rate limit theo cái gì?

Đoạn:

```
request.ip
```

có thể trả:

```
84.123.10.20
```

Rack::Attack sẽ tạo counter gần như:

```
graphql/ip:84.123.10.20
```

IP khác:

```
92.50.20.8
```

có counter khác:

```
graphql/ip:92.50.20.8
```

Cho nên không phải:

```
100 request cho toàn bộ website
```

mà là:

```
100 request / IP / phút
```

9. Request không phải /graphql thì sao?

Ví dụ:

```
GET /health
```

Condition:

```
request.path == "/graphql"
```

là:

```
false
```

Block trả về nil.

Rack::Attack hiểu:

```
rule này không áp dụng
```

10. Response khi bị block

Chúng ta không muốn client nhận response khó hiểu.

Thêm:

```
self.throttled_responder = lambda do |request|
  match_data =
    request.env["rack.attack.match_data"] || {}

  now = match_data[:epoch_time]

  retry_after =
    if now &&
       match_data[:period]
      match_data[:period] -
        (now % match_data[:period])
    end

  headers = {
    "Content-Type" => "application/json"
  }

  headers["Retry-After"] =
    retry_after.to_s if retry_after

  body = {
    error: "Too many requests",
    code: "RATE_LIMITED"
  }

  [
    429,
    headers,
    [body.to_json]
  ]
```

```
end
```

Bây giờ client nhận:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
```

Body:

```
{
  "error": "Too many requests",
  "code": "RATE_LIMITED"
}
```

11. File hoàn chỉnh đầu tiên

Hiện tại:

```
config/initializers/rack_attack.rb
```

có thể là:

```
# frozen_string_literal: true

class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    if request.post? &&
       request.path == "/graphql"
      request.ip
    end
  end

  self.throttled_responder = lambda do |request|
    match_data =
      request.env["rack.attack.match_data"] || {}

    now = match_data[:epoch_time]

    retry_after =
      if now &&
         match_data[:period]
        match_data[:period] -
          (now % match_data[:period])
      end

    headers = {
      "Content-Type" => "application/json"
    }

    headers["Retry-After"] =
      retry_after.to_s if retry_after

    body = {
      error: "Too many requests",
      code: "RATE_LIMITED"
    }

    [
      429,
      headers,
      [body.to_json]
    ]
  end
end
```

Đây đã là rate limiting hoạt động được.

12. Test thực tế — giảm limit xuống 5

Đừng test với:

```
limit: 100
```

Tạm thời đổi:

```
limit: 5,
period: 1.minute
```

Restart Rails:

```
bin/rails server
```

13. Gửi GraphQL request

Ví dụ:

```
query GetMe {
  me {
    id
    email
  }
}
```

Frontend:

```
const query = `
  query GetMe {
    me {
      id
      email
    }
  }
`;
```

Axios:

```
await axios.post(
  "/graphql",
  {
    query,
    operationName: "GetMe"
  },
  {
    withCredentials: true
  }
);
```

14. Tạo button để test spam

React:

```
import axios from "axios";

export default function RateLimitTest() {
  const sendRequests = async () => {
    for (let i = 1; i <= 10; i++) {
      try {
        const response =
          await axios.post(
            "/graphql",
            {
              query: `
                query GetMe {
                  me {
                    id
                    email
                  }
                }
              `,

              operationName: "GetMe"
            },
            {
              withCredentials: true
            }
          );

        console.log(
          `Request ${i}:`,
          response.status
        );
      } catch (error) {
        console.log(
          `Request ${i}:`,
          error.response?.status
        );
      }
    }
  };
```

```
  return (
    <button onClick={sendRequests}>
      Test Rate Limit
    </button>
  );
}
```

Click:

```
Test Rate Limit
```

15. Kết quả

Nếu limit là:

```
limit: 5
```

Console gần như:

```
Request 1: 200
Request 2: 200
Request 3: 200
Request 4: 200
Request 5: 200

Request 6: 429
Request 7: 429
Request 8: 429
Request 9: 429
Request 10: 429
```

Đây là bằng chứng Rack::Attack hoạt động.

16. Frontend phải xử lý 429

Không nên chỉ:

```
console.log(error);
```

Làm:

```
try {
  const response =
    await axios.post(
      "/graphql",
      payload,
      {
        withCredentials: true
      }
    );

  return response.data;
} catch (error) {
  if (error.response?.status === 429) {
    throw new Error(
      "Too many requests. Please try again later."
    );
  }

  throw error;
}
```

UI có thể render:

```
Too many requests.
Please try again later.
```

17. Nhưng rate limit chỉ theo IP có vấn đề

Giả sử công ty có 50 nhân viên.

Tất cả đi qua một corporate network:

```
Public IP:
80.120.10.5
```

User A:

```
30 requests
```

User B:

```
30 requests
```

User C:

```
50 requests
```

Tổng:

```
110
```

Nếu limit:

```
100/IP/minute
```

thì tất cả có thể bị ảnh hưởng.

Vì vậy authenticated GraphQL API thường cần suy nghĩ thêm về:

```
IP limit
+
User limit
```

18. Rate limit theo User

Đây là chỗ cần cẩn thận.

Rack::Attack chạy trước GraphQL resolver, vì vậy nó không tự có:

```
context[:current_user]
```

Không được nghĩ:

```
request.context[:current_user]
```

sẽ có.

Không có.

19. Nếu authentication dựa trên session

Nếu project Rails + Devise của bạn dùng session/cookie, việc rate-limit theo user ở middleware phụ thuộc vào cách authentication/session middleware được bố trí.

Không nên tự viết kiểu:

```
User.find(...)
```

trong mỗi Rack::Attack request.

Ví dụ rất xấu:

```
throttle(...) do |request|
  User.find(
    request.params[:user_id]
  ).id
end
```

Tại sao?

```
request
↓
Rack::Attack
↓
SQL đ tìm User
```

Bạn đang tạo database work ngay trong lớp chống spam.

Attacker spam:

```
10,000 requests
```

→ server vẫn có thể bị ép chạy rất nhiều SQL.

20. Không rate limit bằng userId client gửi

Tuyệt đối không:

```
request.params["userId"]
```

để xác định security identity.

Client có thể gửi:

```
{
  "userId": 1
}
```

rồi request sau:

```
{
  "userId": 2
}
```

rồi:

```
{
  "userId": 3
}
```

Như vậy attacker tự đổi bucket rate limit.

Security identity phải đến từ:

```
trusted authentication
```

không phải GraphQL variables.

21. Cách an toàn cơ bản cho project hiện tại

Cho tầng Rack, chúng ta bắt đầu bằng:

```
IP Rate Limit
```

Ví dụ:

```
throttle(
  "graphql/ip",
  limit: 100,
  period: 1.minute
) do |request|
  request.ip if
    request.post? &&
    request.path == "/graphql"
end
```

Sau này khi authentication architecture rõ ràng, mới thêm:

```
authenticated user limit
```

22. Có thể có nhiều Rate Limit cùng lúc

Ví dụ production:

```
60 requests / minute
+
1000 requests / hour
```

Tại sao?

Một client có thể gửi:

```
59 request/minute
```

liên tục.

Không bao giờ vượt:

```
60/min
```

nhưng một giờ có:

```
3540 requests
```

Do đó có thể dùng hai cửa.

23. Hai throttle

```
class Rack::Attack
  throttle(
    "graphql/ip/minute",
    limit: 60,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
```

```
  end

  throttle(
    "graphql/ip/hour",
    limit: 1000,
    period: 1.hour
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
  end
end
```

Bây giờ:

```
IP
│
├── max 60/minute
│
└── max 1000/hour
```

24. Ví dụ thực tế

User sử dụng dashboard bình thường:

```
08:00 → 10 requests
08:01 → 8 requests
08:02 → 12 requests
```

Không vấn đề.

Bot:

```
08:00:01 → request
08:00:01 → request
08:00:01 → request
...
```

đến:

```
request 61
```

→ 429.

25. Nhưng GraphQL có vấn đề đặc biệt

Giả sử limit:

```
60 request/min
```

Attacker không cần gửi 1000 requests.

Họ gửi chỉ:

```
50 request/min
```

nhưng mỗi request:

```
query Attack {
  projects(first: 100) {
    nodes {
      members {
        user {
          projects(first: 100) {
            nodes {
              tasks(first: 100) {
                nodes {
                  assignee {
                    projects(first: 100) {
                      nodes {
                        id
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
```

```
}
```

Rack::Attack thấy:

```
50 < 60
→ ALLOW
```

Nhưng GraphQL query rất nặng.

Đó chính là lý do Rate Limiting không đủ.

26. Kết hợp Bước 39 + Bước 40

Schema phải có Complexity/Depth protection phù hợp với version graphql-ruby của bạn.

Concept:

```
Request
   │
   ▼
Rack::Attack
   │
   │ 50 requests/min
   │ OK
   ▼
GraphQL
   │
   ├── depth?
   │
   └── complexity?
   │
   ▼
Reject query n□u quá nặng
```

Hai lớp giải quyết hai bài toán khác nhau.

27. Ví dụ Attack số 1 — HTTP Flood

Attacker:

```
POST /graphql
POST /graphql
POST /graphql
...
10,000 l□n
```

Defense:

```
Rack::Attack
```

28. Attack số 2 — Deep Query

Attacker gửi:

```
query {
  projects {
    nodes {
      tasks {
        nodes {
          project {
            tasks {
              nodes {
                project {
                  tasks {
                    nodes {
                      id
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Defense:

```
Depth Limit
```

29. Attack số 3 — Wide / Expensive Query

Query không nhất thiết quá sâu:

```
query {
  projects(first: 100) {
    nodes {
      id
      name
      members {
        id
      }
      tasks(first: 100) {
        nodes {
          id
          title
          description
          status
          priority
        }
      }
    }
  }
}
```

Defense:

```
Complexity Limit
+
Pagination limits
```

30. Attack số 4 — Pagination Abuse

Bạn có:

```
field :tasks,
      Types::TaskType.connection_type,
      null: false
```

Client gửi:

```
query {
  tasks(first: 1000000) {
    nodes {
      id
    }
  }
}
```

Không nên cho phép client tự quyết định số lượng vô hạn.

31. Giới hạn Pagination

Ví dụ:

```
field :tasks,
      Types::TaskType.connection_type,
      null: false,
      max_page_size: 100
```

Bây giờ client không thể hợp pháp lấy:

```
1,000,000 records
```

trong một page.

Concept:

```
Rate Limit
→ s□ request

Complexity
→ chi phí query

Depth
→ độ sâu

max_page_size
→ s□ record/page
```

32. Attack số 5 — Mutation Spam

Rate limiting không chỉ dành cho Query.

Ví dụ mutation:

```
mutation {
  createTask(
    input: {
      projectId: "10"
      attributes: {
        title: "spam"
      }
    }
  ) {
    task {
      id
    }
  }
}
```

Bot gửi:

```
1000 createTask mutations
```

Nếu chỉ rate limit /graphql chung:

```
query
mutation
```

đều bị tính.

Đây đã giúp khá nhiều.

33. Nhưng mutation nhạy cảm có thể cần limit chặt hơn

Ví dụ:

```
login
password reset
invite member
send email
create export
```

Những operation này có thể cần limit riêng.

Vấn đề:

Rack::Attack nhìn HTTP request trước GraphQL execution.

Không nên parse GraphQL document phức tạp bằng regex kiểu:

```
request.body.read.include?(
  "createTask"
)
```

để xây security production.

Nó dễ sai và làm tầng Rack phức tạp.

34. Operation-level Rate Limit

Nếu cần:

```
createTask
→ 20/minute

updateTask
→ 60/minute

dashboard
→ 100/minute
```

thì nên thiết kế rate limiting ở tầng hiểu được:

```
authenticated user
+
GraphQL operation
```

hoặc sử dụng infrastructure/API gateway phù hợp.

Không nên biến Rack::Attack thành GraphQL parser thứ hai.

35. Authentication vẫn bắt buộc

Rate Limit không thay:

```
authenticate_user!
```

Ví dụ:

```
def resolve(**args)
  authenticate_user!

  ...
end
```

Hai thứ khác nhau:

```
Rack::Attack
→ gọi bao nhiêu l□n?

Authentication
→ bạn là ai?
```

36. Authorization vẫn bắt buộc

Ví dụ attacker authenticated.

Họ gọi:

```
mutation {
  deleteProject(
    input: {
      id: "999"
    }
  ) {
    errors
  }
}
```

Rate limit nói:

```
request count OK
```

Nhưng không có nghĩa:

```
user được delete project 999
```

Policy phải kiểm tra:

```
authorize!(project, :destroy)
```

Phân biệt:

```
Rate Limiting
→ frequency

Authentication
→ identity

Authorization
→ permission
```

37. Variables không bao giờ là Authorization

Client gửi:

```
{
  "isAdmin": true
}
```

hoặc:

```
{
  "role": "admin"
}
```

không có giá trị bảo mật.

Server phải lấy:

```
context[:current_user]
```

và Policy quyết định quyền.

38. Error Handling: HTTP vs GraphQL Error

Đây là điểm rất quan trọng.

Khi Rack::Attack block:

```
HTTP 429
```

Response có thể:

```
{
  "error": "Too many requests",
  "code": "RATE_LIMITED"
}
```

Đây không phải GraphQL execution error.

Request chưa đến GraphQL execution.

Trong khi:

```
raise GraphQL::ExecutionError,
      "Forbidden"
```

thường trả GraphQL response:

```
{
  "errors": [
    {
      "message": "Forbidden"
    }
  ]
}
```

Hai tầng hoàn toàn khác nhau.

39. Frontend phải phân biệt

Axios:

```
try {
  const response =
    await axios.post(
      "/graphql",
      payload,
      {
        withCredentials: true
      }
    );

  const result = response.data;

  if (result.errors) {
    console.log(
      "GraphQL error:",
      result.errors
    );
  }

  return result;
} catch (error) {
  if (error.response?.status === 429) {
    console.log(
      "Rate limited"
    );

    return;
  }

  throw error;
}
```

Flow:

```
HTTP 429
→ Axios catch

HTTP 200 + GraphQL errors
→ response.data.errors
```

40. Một helper Axios thực tế

Tạo:

```
src/api/graphqlClient.js
```

Ví dụ:

```
import axios from "axios";

const graphqlClient =
  axios.create({
    baseURL: "/graphql",
    withCredentials: true,
    headers: {
      "Content-Type": "application/json"
    }
  });

export async function graphqlRequest({
  query,
  variables = {},
  operationName
}) {
  try {
    const response =
      await graphqlClient.post(
        "",
        {
          query,
          variables,
          operationName
        }
      );

    if (response.data.errors) {
      throw {
        type: "GRAPHQL_ERROR",
        errors: response.data.errors
      };
    }

    return response.data.data;
  } catch (error) {
    if (error.response?.status === 429) {
      throw {
        type: "RATE_LIMITED",
        message:
          "Too many requests. Please try again later."
      };
    }

    throw error;
  }
}
```

Sau đó component không cần viết lại Axios mỗi lần.

41. Dùng helper

```
const GET_ME = `
  query GetMe {
    me {
      id
      email
    }
  }
`;
```

Gọi:

```
try {
  const data =
    await graphqlRequest({
      query: GET_ME,
      operationName: "GetMe"
    });

  console.log(data.me);
} catch (error) {
  if (
    error.type ===
    "RATE_LIMITED"
```

```
  ) {
    setError(error.message);
  }
}
```

42. Thêm Logging cho Rack::Attack

Bạn nên biết khi server đang block ai.

Có thể subscribe notification của Rack::Attack, tùy version/configuration của gem.

Concept:

```
throttled request
↓
log
↓
IP
path
time
rule
```

Ví dụ log mong muốn:

```
[Rack::Attack]
rule=graphql/ip/minute
ip=84.123.10.20
path=/graphql
status=throttled
```

Không nên log:

```
password
token
session cookie
Authorization header
```

43. Reverse Proxy rất quan trọng

Production thường không phải:

```
Browser
↓
Rails
```

mà có thể:

```
Browser
↓
Cloudflare
↓
Load Balancer
↓
Nginx
↓
Rails
```

Nếu cấu hình proxy sai:

```
request.ip
```

có thể không đại diện cho client như bạn nghĩ.

Do đó khi deploy phải kiểm tra:

```
trusted proxies
forwarded headers
request.ip
```

Không được tin bừa mọi X-Forwarded-For từ Internet.

44. Test request.ip

Development có thể log tạm:

```
Rails.logger.info(
  "Request IP: #{request.ip}"
)
```

Production cần xác nhận:

```
User A
```

```
→ Rails thấy đúng client IP?

User B
→ có IP hợp lý?

Hay mọi người đều thành IP của proxy?
```

Nếu tất cả thành:

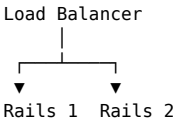
```
10.0.0.5
```

thì rate limit IP sẽ vô tình dùng chung một bucket.

45. Redis và nhiều Rails instances

Đây là vấn đề production quan trọng.

Giả sử bạn deploy:



Nếu counter chỉ nằm local riêng từng process:

```
Rails 1
→ count 40

Rails 2
→ count 40
```

thì toàn hệ thống thực tế đã nhận:

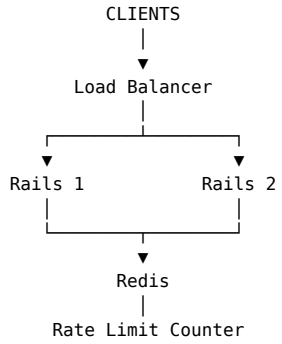
```
80
```

nhưng mỗi instance chỉ thấy:

```
40
```

Vì vậy multi-instance production thường cần shared cache phù hợp, thường là Redis.

46. Kiến trúc



Như vậy mọi Rails instance chia sẻ counter.

47. Ví dụ production thực tế cho project

Giả sử project management API của bạn có:

```
me
projects
project
tasks
dashboard

createProject
updateProject
deleteProject

createTask
updateTask
deleteTask

addProjectMember
```

```
removeProjectMember
```

Một baseline có thể là:

```
GraphQL endpoint
60 requests/min/IP

GraphQL endpoint
1000 requests/hour/IP

GraphQL depth
giới hạn hợp lý

GraphQL complexity
giới hạn hợp lý

Pagination
max 100/page

Authentication
Devise/current_user

Authorization
Policy

N+1
Dataloader
```

Các con số rate limit ở đây là ví dụ để học/test, không phải con số production đúng cho mọi ứng dụng.

48. File Rack::Attack hoàn chỉnh cho bài học

```
# frozen_string_literal: true

class Rack::Attack
  GRAPHQL_PATH = "/graphql"

  throttle(
    "graphql/ip/minute",
    limit: 60,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == GRAPHQL_PATH
  end

  throttle(
    "graphql/ip/hour",
    limit: 1_000,
    period: 1.hour
  ) do |request|
    request.ip if
      request.post? &&
      request.path == GRAPHQL_PATH
  end

  self.throttled_responder = lambda do |request|
    match_data =
      request.env["rack.attack.match_data"] || {}

    period =
      match_data[:period]

    now =
      match_data[:epoch_time]

    retry_after =
      if period && now
        period - (now % period)
      end

    headers = {
      "Content-Type" =>
        "application/json"
    }

    headers["Retry-After"] =
      retry_after.to_s if retry_after

    body = {
```

```
      error: "Too many requests",
      code: "RATE_LIMITED"
    }

    [
      429,
      headers,
      [body.to_json]
    ]
  end
end
```

Vị trí:

```
config/
└── initializers/
    └── rack_attack.rb
```

49. Nhưng khi học hãy bắt đầu đơn giản

Đầu tiên chỉ dùng:

```
throttle(
  "graphql/ip",
  limit: 5,
  period: 1.minute
) do |request|
  request.ip if
    request.post? &&
    request.path == "/graphql"
end
```

Test được:

```
1 → 200
2 → 200
3 → 200
4 → 200
5 → 200
6 → 429
```

Sau khi hiểu rõ mới chuyển thành:

```
minute limit
+
hour limit
+
shared cache
+
monitoring
```

Đừng cấu hình 10 lớp một lúc rồi không biết lớp nào đang block request.

50. Một tình huống thực tế hoàn chỉnh

Giả sử user mở Dashboard.

Frontend gọi:

```
query Dashboard {
  dashboard {
    projectsCount
    tasksCount
    completedTasksCount
  }
}
```

Request:

```
POST /graphql
```

Request bình thường

```
Browser
   │
   ▼
Rack::Attack
   │
   │ 12/60 requests
   ▼
ALLOW
```

```
   |
   ▼
GraphQL
   |
   | depth OK
   | complexity OK
   ▼
authenticate_user!
   |
   ▼
current_user
   |
   ▼
Dashboard Resolver
   |
   ▼
SQL
   |
   ▼
200
```

User double-click/spam hoặc frontend có bug

Ví dụ React useEffect bị loop:

```
useEffect(() => {
  loadDashboard();
});
```

Thiếu:

```
[]
```

Component render:

```
render
→ request
→ setState
→ render
→ request
→ setState
→ render
→ request
...
```

Đây không nhất thiết là attacker.

Nó có thể chỉ là bug frontend.

Sau 60 requests:

```
Rack::Attack
→ 429
```

Server được bảo vệ.

Đây là một ví dụ rất thực tế về lợi ích của rate limiting.

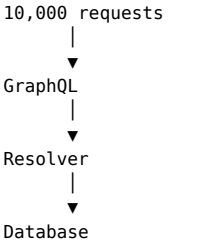
51. Một ví dụ Security Attack thực tế

Attacker đăng nhập bằng account hợp lệ.

Sau đó viết script:

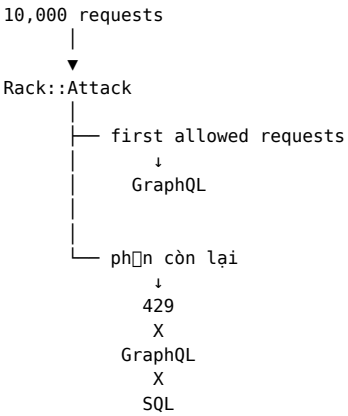
```
for (let i = 0; i < 10000; i++) {
  axios.post(
    "/graphql",
    {
      query: `
        query Dashboard {
          dashboard {
            projectsCount
            tasksCount
          }
        }
      `
    }
  );
}
```

Không có protection:



Database chịu tải.

Có Rack::Attack:



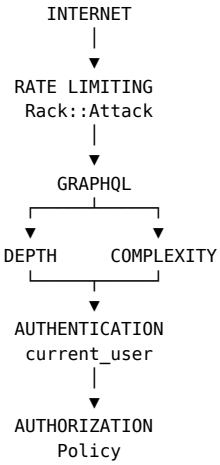
Đây chính là giá trị thực tế của Bước 40.

52. Bảng phân biệt toàn bộ Security

Vấn đề	Protection
10.000 HTTP requests	Rate Limit
Query lồng quá sâu	Depth Limit
Query quá đắt	Complexity Limit
`first: 1000000`	Pagination limit
Chưa login	Authentication
User xóa project người khác	Authorization / Policy
N+1 queries	Dataloader
Input sai kiểu	GraphQL Types / Scalars
Mutation input không hợp lệ	Model / Service validation
Bot spam `/graphql`	Rack::Attack

Không có một mechanism duy nhất giải quyết tất cả.

53. Thứ tự cần nhớ



```
          |
          ▼
BUSINESS LOGIC
     Service
          |
          ▼
      DATABASE
```

54. Bước 39 và Bước 40 khác nhau thế nào?

Bước 39

Hỏi:

> Query này có quá phức tạp không?

```
1 request
↓
query cực lớn
↓
BLOCK
```

Bước 40

Hỏi:

> Client này đang gửi quá nhiều request không?

```
1000 requests
↓
Rack::Attack
↓
BLOCK
```

Production cần cả hai.

55. Checklist Bước 40 cho project

Bạn cần làm theo thứ tự:

1. Thêm gem "rack-attack".
2. Chạy bundle install.
3. Tạo config/initializers/rack_attack.rb.
4. Rate limit POST /graphql theo IP.
5. Tạm đặt limit: 5 để test.
6. Restart Rails.
7. Dùng React button gửi 10 requests.
8. Xác nhận request 6 nhận HTTP 429.
9. Frontend xử lý 429.
10. Sau test đổi sang limit production phù hợp.
11. Giữ Depth + Complexity protection từ Bước 39.
12. Giới hạn pagination.
13. Giữ authentication + Policy.
14. Khi deploy sau proxy, kiểm tra request.ip.
15. Khi chạy nhiều Rails instances, dùng shared cache phù hợp.

Ghi nhớ Bước 40

```
Rack::Attack
→ b□o vệ HTTP endpoint

Rate Limiting
→ giới hạn s□ request

Depth Limit
```

```
→ giới hạn độ sâu GraphQL

Complexity Limit
→ giới hạn chi phí GraphQL

Pagination Limit
→ giới hạn s□ records

Authentication
→ xác định user

Authorization
→ user được phép làm gì
```

Với project của bạn, hình dung một request production như sau:

```
Axios
  ↓
POST /graphql
  ↓
Rack::Attack
  ↓
Depth / Complexity
  ↓
authenticate_user!
  ↓
current_user
  ↓
Policy
  ↓
Resolver / Mutation
  ↓
Service
  ↓
ActiveRecord
  ↓
PostgreSQL
```

Mục đích cốt lõi của Bước 40: một request không được phép tiêu tốn tài nguyên server chỉ vì client có khả năng gửi nó. Rate limit chặn số lượng, Complexity/Depth chặn độ nặng, còn Policy chặn quyền truy cập.